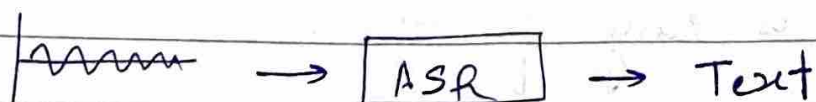


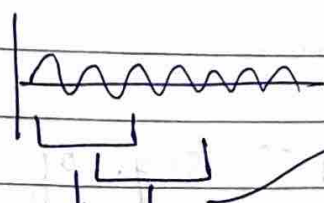
L1

CTC-Recap

- ASR (automatic speech recogniⁿ) or STT (speech to text)



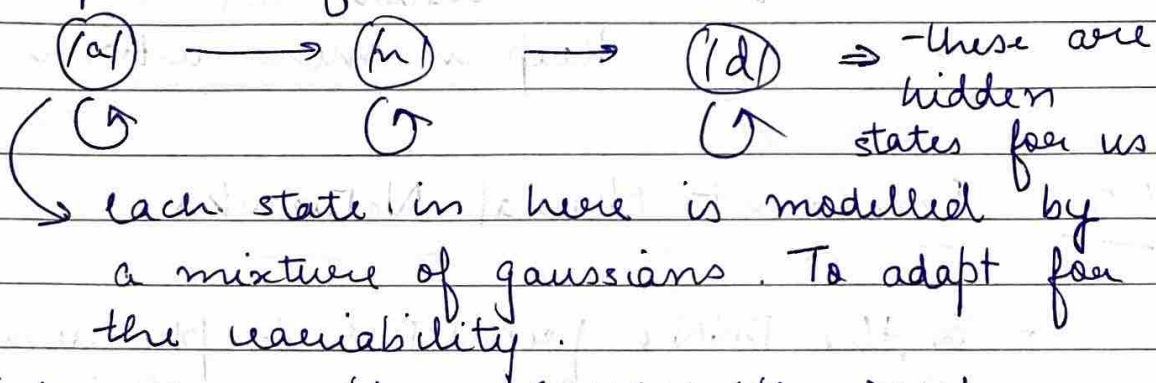
- Traditionally we used, GMM-HMM model, a statistical method.



this is what we observe

O_1 O_2 (captures the property of sound)

HMM captures info about sound states



- Today, we replace GMM with DNN.
DNN-HMM Hybrid model.

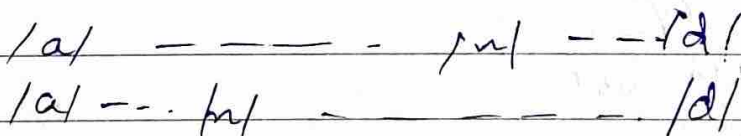
O_1 $O_2 \rightarrow$ MFCC vectors

↓ DNN

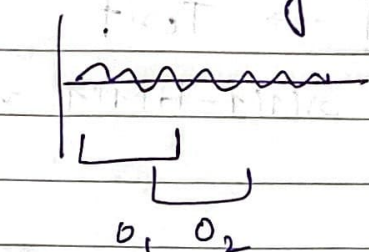
predict what sound

at last layer we have softmax

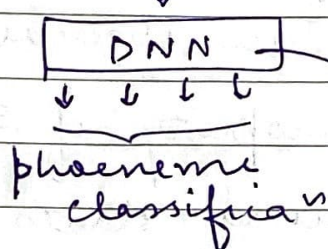
then we saw sth. abt. alignment



- at boundaries, we are not clear. Then, we said let's use CTC $\Rightarrow$ I just want correct sequence (no worry for duration)
- Remember, Indian languages are phonemic. STT is easy for them.



now, lexicon is also needed.



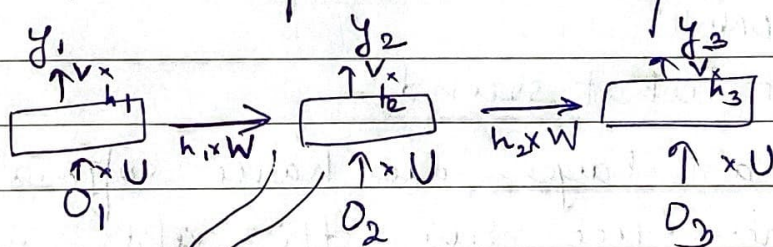
CTC $\rightarrow$ seq. of letters

it is doing frame-by-frame decision. But it doesn't keep in mind what is history.

L2

Recurrent Neural Networks

- so, the DNN's predict each phoneme independent of each other. But the sound we make have some sense. DNN does not keep in mind the history.
- But RNN keeps the history.



RNN

add both input from o_2 & h_1

- _/_/_
- at every layer, everything till now will be remembered.

$$h_3 = f(Wh_2 + UO_3)$$

$$\text{but, } h_2 = f(Wh_1 + UO_2)$$

$$h_3 = f(UO_3 + W f(Wh_1 + UO_2))$$

.... so, on it goes. Thus, history being captured.

$$y_i = O(Vh_i + c) \rightarrow \text{just a bias.}$$

U, V, W , & bias are all shared parameters.

L3

Applications in Language Modeling

- N gram model. $\Rightarrow w_1 \rightarrow w_2 \rightarrow w_3 \rightarrow w_4$

$$\frac{\text{count}(w_1, w_2, w_3, w_4)}{\text{count}(w_1, w_2, w_3)} = P(w_4 | w_1, w_2, w_3)$$

scrap text & then count to find the probability.

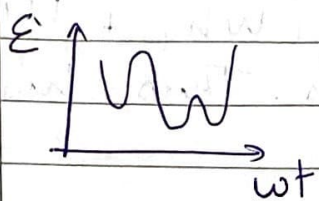
- But those same 5 words wouldn't occur very frequently. Thus, we can't have and calculate very long sentences.
- But, RNN can help us. It will remember history for very long sentences.

L4

Disadvantages with RNN

- first, thing have to represent each word as a number. As computers understand numbers only.

- $W^{new} = W^{old} + \eta \frac{dE}{dW}$ so we have to backpropagate.



- Possibly, - the gradient might go to 0. (Vanishing gradient). Or it becomes too big $\Rightarrow$ then, too big jumps will be taken (exploding vanishing gradient).

- LSTM (Long short-term memory)
 $\hookrightarrow$ use the gates to remember & forget the right amt. of informan. Gates are sigmoid units (0,1)

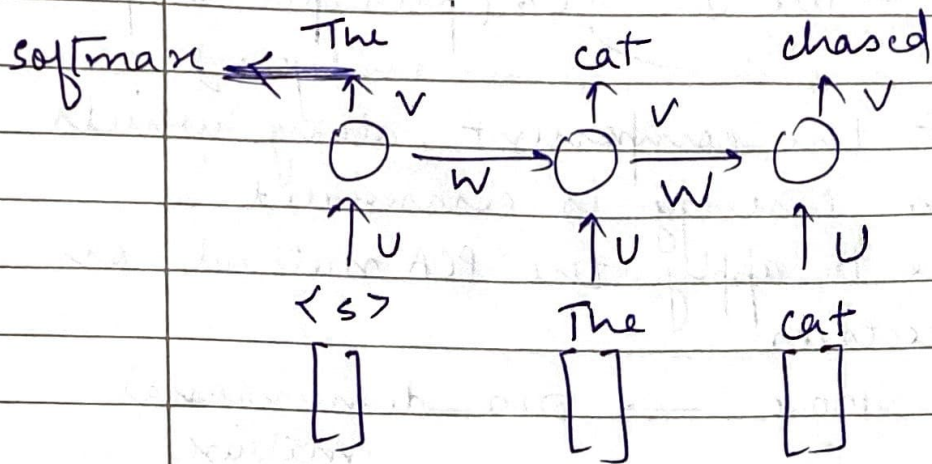
- Next folks, have to feed these text into computers.

L5

Language Model-1

- auto-fill the word, recommendaⁿ.
- acoustic model $\rightarrow$ just looks at sentence
" + LM $\rightarrow$ more reasonable based on entire knowledge base
- then we save the use of RNN to capture the previous context.
- then we solved the problem of RNN using LSTM!

- how to represent the words? text $\rightarrow$ number

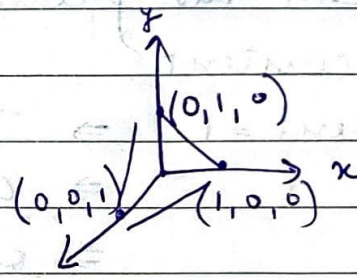


- we can use one-hot representation (all non-zero values, all others are 0).

$$\begin{bmatrix} 1 \\ 0 \\ \vdots \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \\ \vdots \end{bmatrix}, \begin{bmatrix} 0 \\ 0 \\ \vdots \end{bmatrix} \dots$$

$\rightarrow$ it is a very sparse representation, resources wasted.

for 3 word vocab



the distance btw each of the word will be always $\sqrt{2}$. But, there are some words which are more closer to each other than others. No meaningful connections.

L6

LM - 02

- in pattern classification, problem, we have

multiple features $\Rightarrow$ we do dimensionality reducⁿ. $\Rightarrow$ we use PCA (principal component analysis.)

we select the component, along which maximum energy is conserved.

- So, we have to apply the PCA method on one-hot vectors.

$\hookrightarrow$ maybe $100k \rightarrow 500$ dimensional matrix

$$E_{500 \times 1} = W_{500 \times 100,000} \times W_{100,000 \times 1}$$

$\nwarrow$ embedding for i^{th} word $\nwarrow$ embedding matrix

$\hookrightarrow$ we have to estimate it

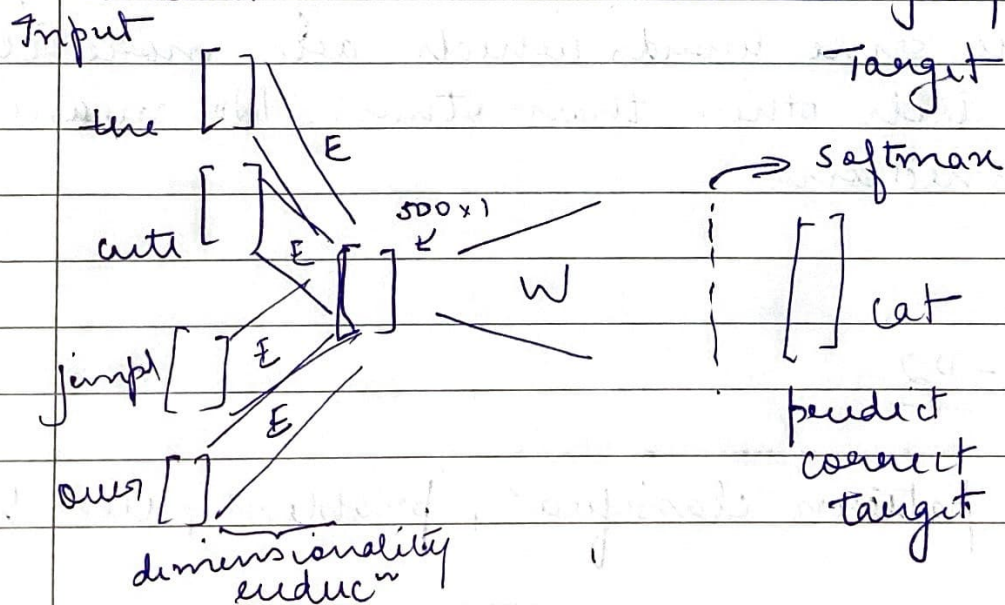
once, using correlaⁿ matrix & eigen analysis.

- Words2vec (self-supervised learning.

$\hookrightarrow$ can we predict a word based on its context?

context = 1 $\Rightarrow$ The _____ cat
 $\Rightarrow$ cute _____ jumped
 $\Rightarrow$ over _____ fox

context = 2 $\Rightarrow$ the cute _____ jumped over



- In word2vec, once "E" embedding matrix is estimated, you can use it to make the sparse one-hot encodings into a dense matrix.
- the distance problem (of $\sqrt{2}$ in one-hot encodings) is also solved ^{by} using the word2vec.